

MODULE 1: BASIC STRUCTURE OF COMPUTERS

BASIC CONCEPTS

- **Computer Architecture (CA)** is concerned with the structure and behaviour of the computer.
- CA includes the information formats, the instruction set and techniques for addressing memory.
- In general covers, CA covers 3 aspects of computer-design namely: 1) Computer Hardware, 2) Instruction set Architecture and 3) Computer Organization.

1. Computer Hardware

- It consists of electronic circuits, displays, magnetic and optical storage media and communication facilities.

2. Instruction Set Architecture

- It is programmer visible machine interface such as instruction set, registers, memory organization and exception handling.

- Two main approaches are 1) CISC and 2) RISC.

(CISC→Complex Instruction Set Computer, RISC→Reduced Instruction Set Computer)

3. Computer Organization

- It includes the high level aspects of a design, such as
 - memory-system
 - bus-structure &
 - design of the internal CPU.
- It refers to the operational units and their interconnections that realize the architectural specifications.
- It describes the function of and design of the various units of digital computer that store and process information.

Q. With a neat diagram describe the functional units of a computer. Give few examples for I/O devices (6 M)

Q.Explain the operation of computer with neat block diagram (10 M)

FUNCTIONAL UNITS

- A computer consists of 5 functionally independent main parts:

- 1) Input
- 2) Memory
- 3) ALU
- 4) Output &
- 5) Control units.

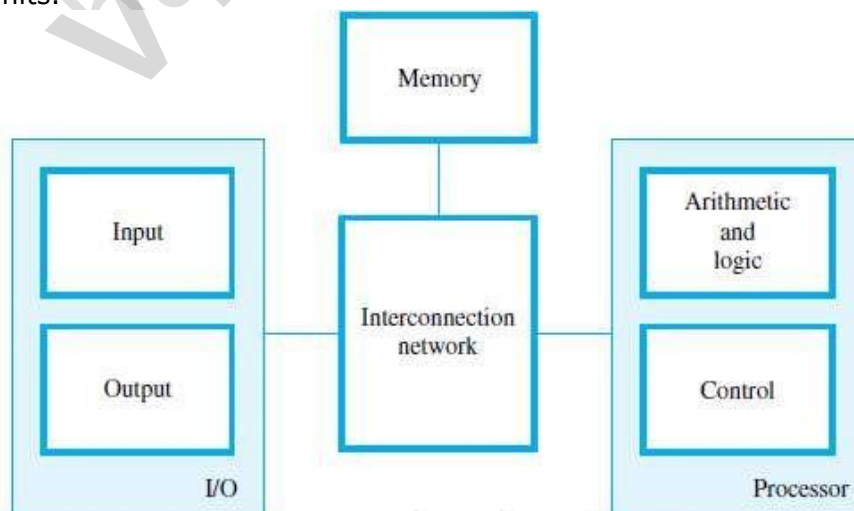


Figure 1.1 Basic functional units of a computer.

Input unit:- Computers accept coded information through input units, which read the data. Whenever a key is pressed, corresponding character or number is translated into its equivalent binary code over a cable & fed either to memory or processor.
Ex: Keyboard, Joysticks, trackballs, mouse, scanners etc are other input devices.

Output unit:- Its basic function is to send the processed results to the outside world.
Examples:- Printer, speakers, monitor etc.

Memory unit: - Its function is to store programs and data. It is basically of two types
1. Primary memory 2. Secondary memory

1. **Primary memory:** - The programs which has to be executed must be stored in this memory. Each memory location stores data in it. Each memory location is identified by its address.
Examples:- RAM, ROM, CACHE etc..

2 **Secondary memory:** - It is used where large amounts of data & programs have to be stored, particularly information that is accessed infrequently.
Examples: - Magnetic disks & tapes, optical disks (ie CD-ROM's), floppies etc.,

Arithmetic logic unit (ALU):- This unit performs arithmetic operations like addition, subtraction, division, multiplication and logical operations like not, and, or, xor etc. on the operands(data) that are brought into the ALU from memory. Then according to the instructions the operation is performed in the required sequence.

Control unit:- The actual timing signals that govern the transfer of data between input unit, processor, memory and output unit are generated by the control unit.

BASIC OPERATIONAL CONCEPTS

- An Instruction consists of 2 parts, 1) Operation code (Opcode) and 2) Operands.

OPCODE	OPERANDS
--------	----------

- The data/operands are stored in memory.
- The individual instruction are brought from the memory to the processor.
- Then, the processor performs the specified operation.
- Let us see a typical instruction
ADD LOCA, R0
- This instruction is an addition operation. The following are the steps to execute the instruction: Step 1: Fetch the instruction from main-memory into the processor.
Step 2: Fetch the operand at location LOCA from main-memory into the processor.
Step 3: Add the memory operand (i.e. fetched contents of LOCA) to the contents of register R0. Step 4: Store the result (sum) in R0.
- The same instruction can be realized using 2 instructions as:
Load LOCA, R1
Add R1, R0
- The following are the steps to execute the instruction:
Step 1: Fetch the instruction from main-memory into the processor.
Step 2: Fetch the operand at location LOCA from main-memory into the register R1. Step 3: Add the content of Register R1 and the contents of register R0.
Step 4: Store the result (sum) in R0.

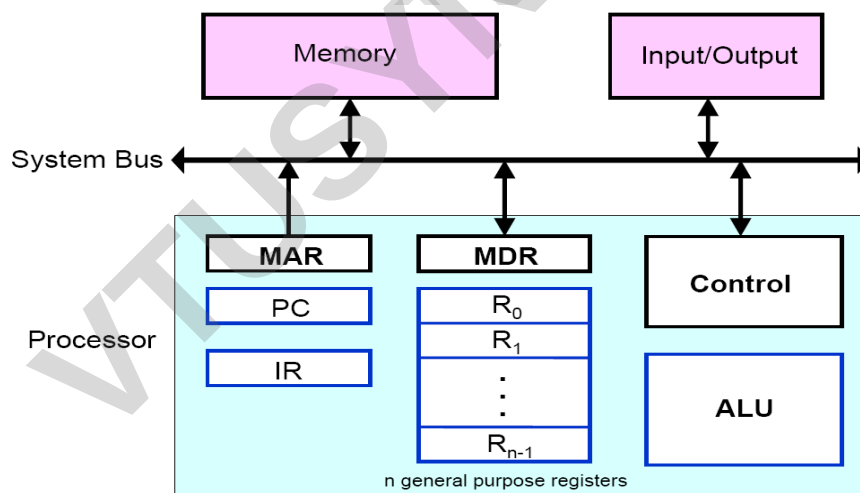
Q. With a neat diagram, discuss the operational concepts in a computer highlight the role of PC, MAR, MDR and IR – (08 M)

Various Processor Registers

- The **processor** contains ALU, control-circuitry and many registers.
- The processor has 'n' general-purpose registers **R₀** through **R_{n-1}**.
- The **IR (Instruction Register)** holds the instruction that is currently being executed.
- The **control-unit** generates the timing-signals that determine when a given action is to take place.
- The **PC (Program Counter)** contains the memory-address of the next-instruction to be fetched & executed.
- During the execution of an instruction, the contents of PC are updated to point to next instruction.
- The **MAR (Memory Address Register)** holds the address of the memory-location to be accessed.
- The **MDR (Memory Data Register)** contains the data to be written into or read out of the addressed location.

STEPS TO EXECUTE AN INSTRUCTION between processor and memory

- 1) The address of first instruction (to be executed) gets loaded into PC.
- 2) The contents of PC (i.e. address) are transferred to the MAR & control-unit issues Read signal to memory.
- 3) After certain amount of elapsed time, the first instruction is read out of memory and placed into MDR.
- 4) Next, the contents of MDR are transferred to IR. At this point, the instruction can be decoded & executed.
- 5) To fetch an operand, it's address is placed into MAR & control-unit issues Read signal. As a result, the operand is transferred from memory into MDR, and then it is transferred from MDR to ALU.
- 6) Likewise required number of operands is fetched into processor.
- 7) Finally, ALU performs the desired operation.
- 8) If the result of this operation is to be stored in the memory, then the result is sent to the MDR.
- 9) The address of the location where the result is to be stored is sent to the MAR and a Write cycle is initiated.
- 10) At some point during execution, contents of PC are incremented to point to next instruction in the program.



Q. Explain single bus and multiple bus structure in computer (06 M)

BUS STRUCTURE

- A bus is a group of lines that serves as a connecting path for several devices.
- A bus may be lines or wires.
- The lines carry data or address or control signal.
- There are 2 types of Bus structures: 1) Single Bus Structure and 2) Multiple Bus Structure.

1) Single Bus Structure

- Because the bus can be used for only one transfer at a time, only 2 units can actively use the bus at any given time.
- Bus control lines are used to arbitrate multiple requests for use of the bus.

➤ **Advantages:**

- 1) Low cost &
- 2) Flexibility for attaching peripheral devices.

2) Multiple Bus Structure

- Systems that contain multiple buses achieve more concurrency in operations.
- Two or more transfers can be carried out at the same time.
- **Advantage:** Better performance.
- **Disadvantage:** Increased cost.

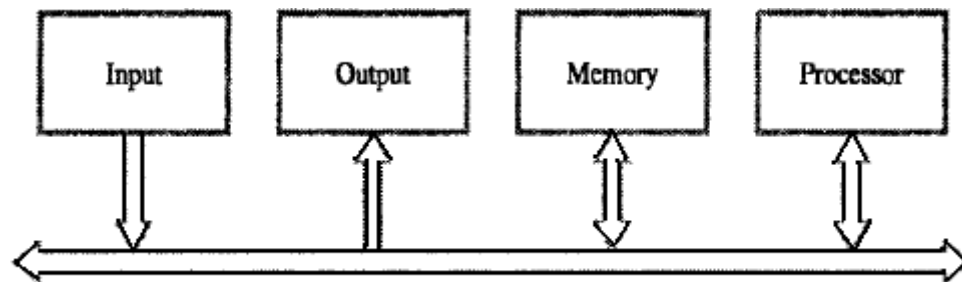


Figure 1.3 Single-bus structure.

- The devices connected to a bus vary widely in their speed of operation.
- To synchronize their operational-speed, buffer-registers can be used.

• **Buffer Registers**

- are included with the devices to hold the information during transfers.
- prevent a high-speed processor from being locked to a slow I/O device during data transfers.

Memory mapped I/O

It is an usual assumption that the addresses issued by the processor to access instructions and operands always refer to memory locations. Many computers use an arrangement called memory mapped I/O in which some memory address values are used to refer to peripheral device buffer registers, such as DATAIN and DATAOUT. Thus no special instructions are needed to access the contents of these registers; data can be transferred between these registers and the processor using instructions such as Move, Load or Store.

For Example, the contents of the keyboard character buffer DATAIN can be transferred to register R1 in the processor by the instruction

MoveByte DATAIN,R1

Similarly, the contents of register R1 can be transferred to DATAOUT by the instruction

MoveByte R1, DATAOUT

The status flags SIN and SOUT are automatically cleared when the buffer registers DATAIN and DATAOUT are referenced, respectively. The MoveByte operation code signifies that the operand size is a byte, to distinguish it from the operation code Move that has been used for word operands.

Q. Explain system Software functions in computer(06M).

- In order for a user to enter and run an application program, the computer must already contain some system software in its memory.
- System software is a collection of programs that are executed as needed to perform functions such as
 - Receiving and interpreting user commands
 - Running standard application programs such as word processors, etc, or games
 - Managing the storage and retrieval of files in secondary storage devices
 - Controlling I/O units to receive input information and produce output results
 - Translating programs from source form prepared by the user into object form consisting of machine instructions
 - Linking and running user-written application programs with existing standard library routines, such as numerical computation packages

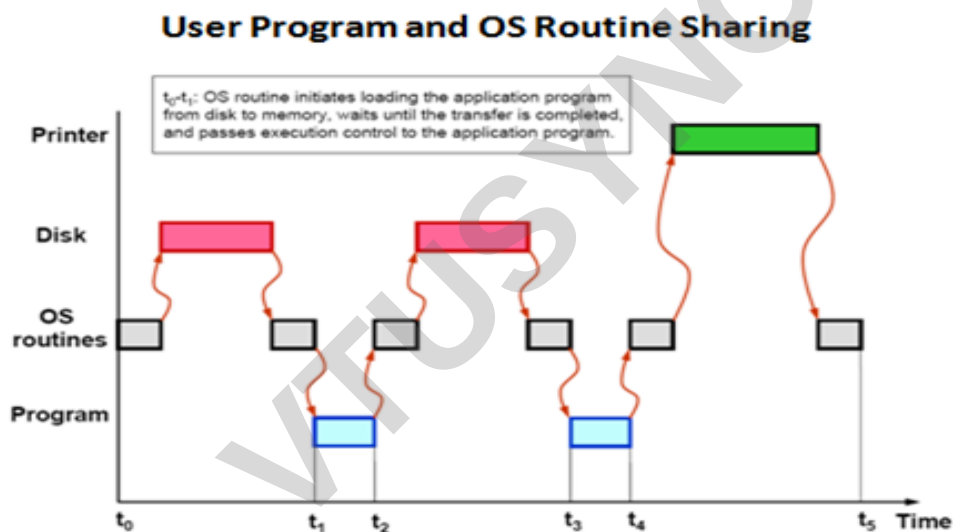
- System software is thus responsible for the coordination of all activities in a computing system

Q. What is operating system? Explain user program and OS routine sharing the processor (08 M).

- Operating system (OS)
 - This is a large collection of routines, that controls the sharing and interaction among various computer units as they perform application programs
- The OS routines perform the tasks required to assign computer resource to individual application programs
 - These tasks include assigning memory and magnetic disk space to program and data files, moving data between memory and disk units, and handling I/O operations
- In the diagram, a system with one processor, one disk, and one printer is given to explain the basics of OS
 - Assume that part of the program's task involves reading a data file from the disk into the memory, performing some computation on the data, and printing the results.

In the diagram shown, the above procedure as per to the time slots is explained.

- In the time period $t_0 - t_1$, **Operating system loads the application program from disk to memory.**
- In the time period $t_1 - t_2$ **program is executed**
- t_2-t_3 **program accesses disk**
- $t_3 - t_4$ **program executes some more**
- t_4-t_5 **program accesses printer**
- At t_5 **program terminates**



PERFORMANCE

- The most important measure of performance of a computer is how quickly it can execute programs.
- The speed of a computer is affected by the design of
 - 1) Instruction-set.
 - 2) Hardware & the technology in which the hardware is implemented.
 - 3) Software including the operating system.
- Because programs are usually written in a HLL(HLL→ High Level Language), performance is also affected by the compiler that translates programs into machine language.

- For best performance, it is necessary to design the compiler, machine instruction set and hardware in a co-ordinated way.

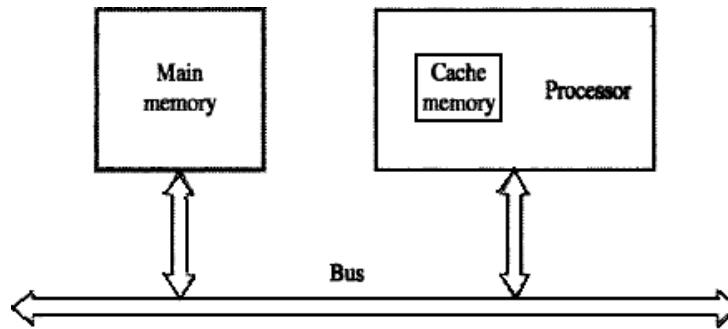


Figure 1.5 The processor cache.

- Let us examine the flow of program instructions and data between the memory & the processor.
- At the start of execution, all program instructions are stored in the main-memory.
- As execution proceeds, instructions are fetched into the processor, and a copy is placed in the cache.
- Later, if the same instruction is needed a second time, it is read directly from the cache.

Q. Explain in brief different types of key parameters that affect the processor performance (05 M)

Factors for improving or affecting the performance

1. PROCESSOR CLOCK

- Processor circuits are controlled by a timing signal called a **Clock**.
- The clock defines regular time intervals called **Clock Cycles**.
- To execute a machine instruction, the processor divides the action to be performed into a sequence of basic steps such that each step can be completed in one clock cycle.
- Let P = Length of one clock cycle
R = Clock rate.
- Relation between P and R is given by $R = \frac{1}{P}$
- R is measured in cycles per second.
- Cycles per second is also called Hertz (Hz)

2. BASIC PERFORMANCE EQUATION (Q. Explain computer basic performance equation -04 M)

- Let T = Processor time required to executed a program.
- N = Actual number of instruction executions.
- S = Average number of basic steps needed to execute one machine instruction.
- R = Clock rate in cycles per second.
- The program execution time is given by

$$T = \frac{N \times S}{R} \quad \text{-----(1)}$$

- Equ1 is referred to as the basic performance equation.
- To achieve high performance, the computer designer must reduce the value of T, which means reducing N and S, and increasing R.
 - The value of N is reduced if source program is compiled into fewer machine instructions.
 - The value of S is reduced if instructions have a smaller number of basic steps to perform.
 - The value of R can be increased by using a higher frequency clock.
- Care has to be taken while modifying values since changes in one parameter may affect the other.

3. CLOCK RATE

- There are 2 possibilities for increasing the clock rate R:
 - 1) Improving the IC technology makes logic-circuits faster.
 - 2) Reducing the amount of processing done in one basic step also reduces the clock period P.

4. Compiler:

- A compiler translates a high-level language program into a sequence of machine instructions.
- To reduce N, we need a suitable machine instruction set and a compiler that makes good use of it.
- Goal is to reduce $N \times S$
 - N= is the actual number of instruction execution
 - S= is the total number of basic steps needed to execute one machine cycle instruction.

5. Pipelining and superscalar operation.

- Pipelining: by overlapping the execution of successive instructions
- Superscalar: different instructions are concurrently executed with multiple instruction pipelines. This means that multiple functional units are needed. This means that multiple functional units are used creating parallel paths through which different instructions can be executed in parallel with such an arrangement, it becomes possible to start the execution of several instructions in every clock cycle. This mode of operation is called superscalar execution.

6. CISC and RISC

- It is much easier to implement efficient pipelining in processor with simple instruction sets
- Reduced Instruction Set Computers (RISC)
- Complex Instruction Set Computers (CISC)

INSTRUCTION SET: CISC AND RISC

RISC	CISC
Simple instructions taking one cycle.	Complex instructions taking multiple cycle.
Instructions are executed by hardwired control unit.	Instructions are executed by microprogrammed control unit.
Few instructions.	Many instructions.
Fixed format instructions.	Variable format instructions.
Few addressing modes, and most instructions have register to register addressing mode.	Many addressing modes.
Multiple register set.	Single register set.
Highly pipelined.	No pipelined or less pipelined.

Problem 1:

List the steps needed to execute the machine instruction:

Add R2, LOCA

in terms of transfers between the components of processor and some simple control commands.
Assume that the address of the memory-location containing this instruction is initially in register PC.

Solution:

1. Transfer the contents of register PC (address of this instruction) to register MAR.
2. Issue a Read command to memory.
And, then wait until the instruction is transferred into register MDR.

3. Transfer the instruction from MDR into IR(Instruction Register) and decode it.
4. Transfer the address LOCA from IR to MAR.
5. Issue a Read command and wait until MDR is loaded with data present at LOCA address.
6. Transfer contents of MDR to the ALU.
7. Transfer contents of R2 to the ALU.
8. Perform addition of the two operands in the ALU and transfer result at location LOCA.
9. Transfer contents of PC to ALU.
10. Add 1 to operand in ALU and transfer incremented address to PC.

Problem 2:

List the steps needed to execute the machine instruction:

Add R4, R2, R3

in terms of transfers between the components of processor and some simple control commands.

Assume that the address of the memory-location containing this instruction is initially in register PC.

Solution:

1. Transfer the contents of register PC to register MAR.
2. Issue a Read command to memory.
And, then wait until the instruction is transferred into register MDR.
3. Transfer the instruction from MDR into IR and decode it.
4. Transfer contents of R1 and R2 to the ALU.
5. Perform addition of two operands in the ALU and transfer answer into R3.
6. Transfer contents of PC to ALU.
7. Add 1 to operand in ALU and transfer incremented address to PC.

Problem 3:

(a) Given a short sequence of machine instructions for the task "Add the contents of memory-location A to those of location B, and place the answer in location C". Instructions:

Load LOC, R_i and
Store R_i, LOC

are the only instructions available to transfer data between memory and the general purpose registers. Add instructions do not change contents of either location A or B.

(b) Suppose that Move and Add instructions are available with the formats:

Move Location1, Location2 and
Add Location1, Location2

These instructions move or add a copy of the operand at the second location to the first location, overwriting the original operand at the first location. Either or both of the operands can be in the memory or the general-purpose registers. Is it possible to use fewer instructions of these types to accomplish the task in part (a)? If yes, give the sequence.

Solution:

(a)

Load A, R0
Load B, R1
Add R0, R1
Store R1, C

(b) Yes;

Move B, C
Add A, C

Problem 4:

A program contains 1000 instructions. Out of that 25% instructions requires 4 clock cycles, 40% instructions requires 5 clock cycles and remaining require 3 clock cycles for execution. Find the total time required to execute the program running in a 1 GHz machine.

Solution:

N = 1000

25% of N= 250 instructions require 4 clock cycles.

40% of N =400 instructions require 5 clock cycles. 35% of N=350 instructions require 3 clock cycles.

$$T = (N \times S) / R = (250 \times 4 + 400 \times 5 + 350 \times 3) / 1 \times 10^9 = (1000 + 2000 + 1050) / 1 \times 10^9 = 4.05 \mu s.$$

Problem 5:

For the following processor, obtain the performance.

Clock rate = 800 MHz

No. of instructions executed = 1000

Average no of steps needed / machine instruction = 20

Solution:

$$T = \frac{N \times S}{R} = (1000 \times 20) / 800 \times 10^6 = 25 \text{ micro sec or } 25 \times 10^{-6} \text{ sec}$$
